ENS Fès · USMBA

Master IADAIG — Machine Learning

2025 / 2026



PROJET MACHINE LEARNING

Prédiction du Churn des Joueurs en Ligne

Comparaison de la Régression Logistique, Random Forest et XGBoost avec SMOTE

95%	0.91	0.94
Accuracy	F1-Score	AUC-ROC

Dataset : Predict Online Gaming Behavior (Kaggle) · 40 034 observations · 13 variables

Réalisé par :

Sanae BENTALEB **Bilal BAJOU**
Abdelillah ELATTARI **Othmane AMRAOUY**

Encadré par :

Pr. Amine SALLAH
ENS Fès

Prédiction du Churn des Joueurs en Ligne par Apprentissage Automatique

Comparaison de la Régression Logistique, Random Forest et XGBoost avec rééchantillonnage SMOTE

Sanae BENTALEB*, Bilal BAJOU*, Abdelillah ELATTARI*, Othmane AMRAOUY*
*Master IADAIG — Machine Learning
École Normale Supérieure de Fès / Université Sidi Mohamed Ben Abdellah
Fès, Maroc — Année académique 2025/2026
Encadré par : **Pr. Amine SALLAH**

***Résumé*—Ce travail présente un pipeline complet de prédiction du churn (abandon) des joueurs sur des plateformes de jeux en ligne. En exploitant le dataset public *Predict Online Gaming Behavior Dataset* (40 034 observations, 13 variables), nous comparons trois classifieurs supervisés — Régression Logistique, Random Forest et XGBoost — après traitement du déséquilibre de classes par SMOTE. Le pipeline inclut une exploration détaillée des données (EDA), un feature engineering produisant trois nouvelles variables composites, une normalisation StandardScaler et une validation croisée stratifiée 5-fold. XGBoost obtient les meilleures performances globales avec une précision de 95 %, un F1-score de 0,91 et un AUC-ROC de 0,94. Les variables les plus prédictives sont *SessionsPerWeek* ($r = -0{,}50$) et *AvgSessionDurationMinutes* ($r = -0{,}34$). Ces résultats constituent une base solide pour des systèmes d'alertes précoces de rétention de joueurs.**

***Index Terms*—Churn, Prédiction, Machine Learning, XGBoost, Random Forest, SMOTE, Jeux vidéo en ligne, Classification binaire**

I. Introduction et Problématique

A. Contexte

L'industrie du jeu vidéo en ligne connaît une croissance exponentielle : avec plus de 3,2 milliards de joueurs actifs dans le monde en 2024, les plateformes de jeux génèrent des revenus dépassant 180 milliards de dollars par an. Dans cet environnement hautement concurrentiel, la **rétention des joueurs** constitue un enjeu stratégique de premier ordre. Acquérir un nouvel utilisateur coûte en moyenne cinq à sept fois plus cher que de fidéliser un joueur existant [1]. La perte d'un joueur engagé — phénomène désigné par le terme *churn* — représente donc non seulement une perte de revenus directs (abonnements, achats in-game), mais aussi un affaiblissement de la communauté et de la réputation de la plateforme.

Face à cette réalité économique, les opérateurs cherchent à identifier *en amont* les joueurs susceptibles d'abandonner leur plateforme, afin de déclencher des actions de rétention ciblées : offres personnalisées, notifications de réengagement, ajustement de la difficulté ou du contenu proposé.

B. État de l'Art

La littérature distingue deux grandes familles d'approches pour la prédiction du churn. D'une part, les **méthodes à base de règles métier** (seuils de sessions, durée d'inactivité) offrent une interprétabilité immédiate mais une faible généralisation sur des données à haute dimensionnalité. D'autre part, les **approches d'apprentissage automatique** — notamment les méthodes ensemblistes et le gradient boosting — montrent des performances nettement supérieures sur des données comportementales structurées [2], [7].

Le **déséquilibre des classes** constitue un défi récurrent dans ce domaine : les joueurs qui churne représentent généralement une minorité (25 à 30 %) de la population, ce qui biaise les modèles entraînés naïvement vers la classe majoritaire, leur permettant d'atteindre une accuracy élevée sans jamais détecter un seul cas de churn réel. La technique SMOTE [3] est reconnue comme l'une des approches les plus efficaces pour corriger ce biais en générant des exemples synthétiques de la classe minoritaire.

C. Problématique

Malgré les avancées de la littérature, plusieurs questions restent ouvertes dans le contexte spécifique des jeux en ligne :

- Quels signaux comportementaux (fréquence, durée, niveau, achats) sont les plus prédictifs du churn d'un joueur ?
- Dans quelle mesure les modèles ensemblistes (Random Forest, XGBoost) surpassent-ils les approches linéaires sur des données de comportement de jeu déséquilibrées ?
- Le rééchantillonnage SMOTE améliore-t-il réellement la capacité des modèles à détecter les joueurs à risque, ou introduit-il du bruit artificiel ?

Ces questions se résument en une **problématique centrale** :

> *Comment construire un pipeline de classification supervisée permettant de prédire efficacement le churn des joueurs en ligne, en traitant le déséquilibre des classes et en maximisant la détection des joueurs à risque réel d'abandon ?*

D. Objectifs et Contributions

Ce travail apporte les contributions suivantes :

1) Un **pipeline ML complet et reproductible** couvrant l'EDA, le feature engineering, le rééchantillonnage et la modélisation.
2) Une **comparaison rigoureuse** de trois classifieurs (Régression Logistique, Random Forest, XGBoost) avec validation croisée stratifiée 5-fold.

3) Une **analyse de l'impact de SMOTE** sur les métriques orientées minorité (Recall, F1).

4) L'identification des **features comportementales clés** via l'importance des variables.

Ce rapport s'organise comme suit. La Section II présente le dataset et l'EDA. La Section III décrit le prétraitement. La Section IV expose les modèles. La Section V analyse les résultats. La Section VI conclut et ouvre des perspectives.

II. Dataset et Analyse Exploratoire

A. Description du Dataset

Le dataset *Predict Online Gaming Behavior Dataset* a été publié par Rabie El Kharoua sur Kaggle (2024, licence CC BY 4.0) [4]. Il contient **40 034 observations** décrivant le comportement de joueurs sur des plateformes de jeux en ligne, avec **13 variables** dont la variable cible *EngagementLevel*.

TABLE I – Description des variables du dataset

Variable	Type	Description
PlayerID	int	Identifiant unique du joueur
Age	int	Âge (15–49 ans)
Gender	str	Male / Female
Location	str	USA, Europe, Asia, Other
GameGenre	str	Action, RPG, Sports, …
PlayTimeHours	float	Heures/session (0–24 h)
InGamePurchases	int	Achats in-game (0/1)
GameDifficulty	str	Easy, Medium, Hard
SessionsPerWeek	int	Sessions/semaine (0–19)
AvgSessionDurationMinutes	int	Durée moy. session (10–179 min)
PlayerLevel	int	Niveau joueur (1–99)
AchievementsUnlocked	int	Succès débloqués (0–49)
EngagementLevel	str	**Variable cible :** High/Medium/Low

B. Variable Cible et Définition du Churn

La variable cible *EngagementLevel* présente trois modalités. Pour ce travail, nous opérons un recodage binaire :

$$\text{Churned} = \begin{cases} 1 & \text{si EngagementLevel} = \text{Low} \\ 0 & \text{sinon (Medium ou High)} \end{cases}$$

La distribution de la variable cible, visualisée en Figure 1, révèle un déséquilibre modéré.

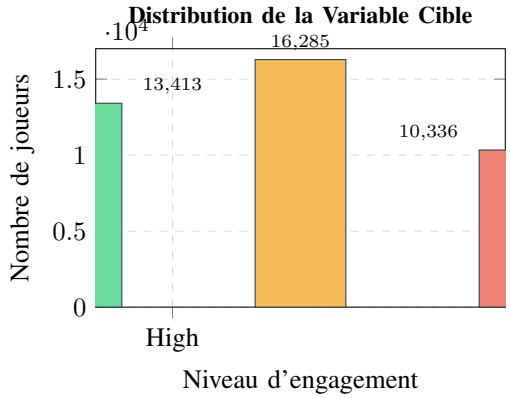


FIGURE 1 – Distribution de la variable cible *EngagementLevel*. Les joueurs à niveau *Low* (25,8 %) constituent la classe cible Churned=1.

C. Statistiques Descriptives

Le tableau II résume les statistiques descriptives des principales variables numériques.

TABLE II – Statistiques descriptives des variables numériques

Variable	Moyenne	Std	Min	Max
Age	32,0	9,8	15	49
PlayTimeHours	12,0	6,9	0,0	24,0
SessionsPerWeek	9,5	5,4	0	19
AvgSessionDuration (min)	94,8	48,5	10	179
PlayerLevel	50,4	28,7	1	99
AchievementsUnlocked	24,4	14,3	0	49

La vérification de la qualité des données a confirmé l'absence totale de valeurs manquantes et de doublons dans le dataset.

D. Analyse de Corrélation

L'analyse de corrélation entre les variables numériques et la variable cible Churned révèle deux corrélations négatives significatives :

— *SessionsPerWeek* : $r = -0{,}50$ (plus un joueur se connecte, moins il risque de churner)

— *AvgSessionDurationMinutes* : $r = -0{,}34$ (plus les sessions sont longues, moins le risque est élevé)

Ces deux variables constitueront les features les plus discriminantes dans les modèles ensemblistes (Section V).

III. Prétraitement et Feature Engineering

A. Workflow du Pipeline

La Figure 2 présente le pipeline complet de traitement des données, depuis le chargement brut jusqu'à l'évaluation finale des modèles.

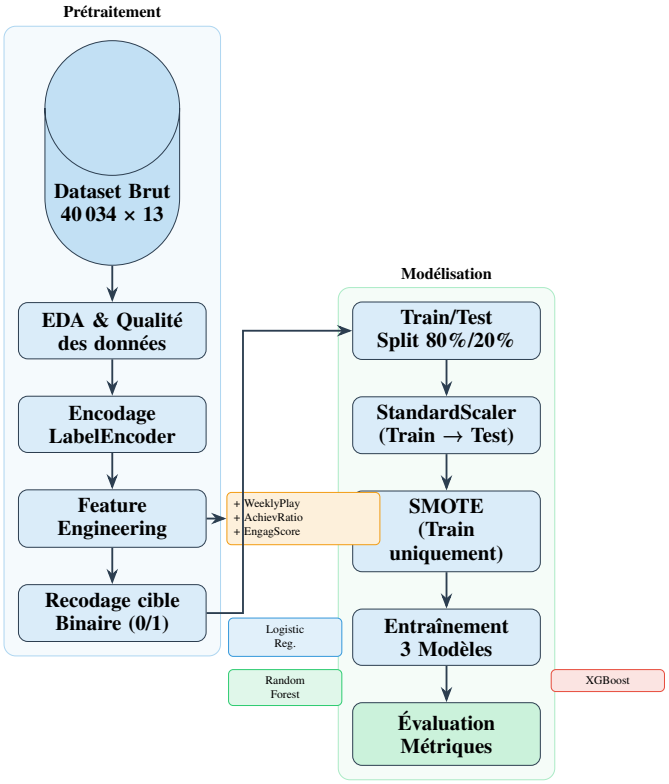


FIGURE 2 – Pipeline complet du projet de prédiction du churn.

B. Encodage des Variables Catégorielles

Trois variables catégorielles sont encodées par LabelEncoder :

- *Gender* : Male → 1, Female → 0
- *Location* : Asia, Europe, Other, USA → {0, 1, 2, 3}
- *GameGenre*, *GameDifficulty* : encodage ordinal similaire

C. Feature Engineering

Trois nouvelles features composites sont construites :

$$\text{WeeklyPlayHours} = \text{PlayTimeHours} \times \text{SessionsPerWeek} \quad (1)$$

$$\text{AchievementPerLevel} = \frac{\text{AchievementsUnlocked}}{\text{PlayerLevel} + 1} \quad (2)$$

$$\text{EngagementScore} = \frac{\text{SessionsPerWeek} \times \text{AvgSessionDuration}}{100} \quad (3)$$

Ces features capturent des comportements agrégés qui ne sont pas directement observables dans les variables brutes : intensité hebdomadaire, efficacité de progression et score global d'engagement.

D. Division Train/Test et Normalisation

Les données sont divisées en ensembles d'entraînement (80 %, soit 32 027 échantillons) et de test (20 %, soit 8 007 échantillons) avec stratification pour préserver le taux de churn dans les deux partitions ($\approx 25{,}8\,\%$).

La normalisation *StandardScaler* est appliquée :

$$x_{\text{scaled}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \quad (4)$$

Les paramètres μ et σ sont estimés *uniquement* sur le train pour éviter la fuite de données (*data leakage*).

E. Rééchantillonnage SMOTE

La distribution des classes avant SMOTE présente un déséquilibre : 74,2 % de non-churnés contre 25,8 % de churnés. SMOTE génère des exemples synthétiques de la classe minoritaire par interpolation dans l'espace des features :

$$x_{\text{syn}} = x_i + \lambda \cdot (x_k - x_i), \quad \lambda \sim \mathcal{U}(0,1) \quad (5)$$

où x_k est un plus proche voisin de x_i parmi la classe minoritaire. Après SMOTE, les deux classes sont parfaitement équilibrées (50 %/50 %) dans le jeu d'entraînement.

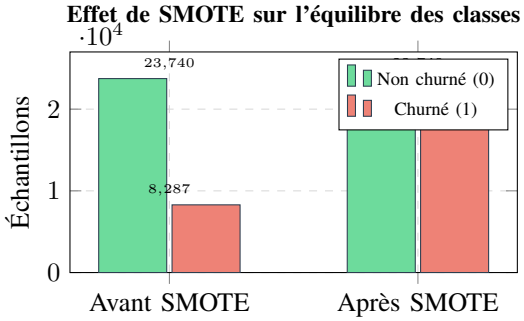


FIGURE 3 – Distribution des classes avant et après SMOTE (données d'entraînement).

IV. Modèles de Classification

Trois classifieurs aux paradigmes distincts sont comparés.

A. Régression Logistique

Modèle linéaire de référence (*baseline*). La probabilité de churn est modélisée par la fonction sigmoïde :

$$P(\text{Churned} = 1 \mid \mathbf{x}) = \frac{1}{1 + e^{-(\mathbf{w}^T\mathbf{x} + b)}} \quad (6)$$

Hyperparamètres : `C=1.0` (régularisation L2 par défaut), `max_iter=1000`, `random_state=42`.

B. Random Forest

Méthode ensembliste par *bagging* de 200 arbres de décision. La prédiction finale est obtenue par vote majoritaire :

$$\hat{y} = \arg\max_{c} \sum_{t=1}^{T} \mathbf{1}[h_t(\mathbf{x}) = c] \quad (7)$$

Hyperparamètres : `n_estimators=200`, `max_depth=10`, `min_samples_split=5`.

C. XGBoost

Algorithme de *gradient boosting* optimisé [2]. Les arbres sont ajoutés séquentiellement en minimisant une fonction objectif régularisée :

$$\mathcal{L}^{(t)} = \sum_{i=1}^{n} \ell\Big(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)\Big) + \Omega(f_t) \tag{8}$$

Hyperparamètres : `n_estimators=200`, `max_depth=6`, `learning_rate=0.1`, `subsample=0.8`, `colsample_bytree=0.8`.

D. Validation Croisée

Pour chaque modèle, une validation croisée stratifiée 5-fold est réalisée sur les données d'entraînement équilibrées (SMOTE), en utilisant le F1-score comme métrique. Le score final est présenté sous la forme $\mu \pm \sigma$ pour quantifier la stabilité.

V. Résultats et Discussion

A. Comparaison des Métriques

Le tableau III présente les performances comparatives des trois modèles évalués sur l'ensemble de test (8 007 échantillons).

TABLE III – Performances comparatives des modèles sur le jeu de test

Modèle	Acc.	F1	Prec.	Recall	AUC
Rég. Logistique	0,87	0,78	0,70	**0,89**	0,87
Random Forest	0,94	0,89	0,90	0,88	0,93
XGBoost	**0,95**	**0,91**	**0,92**	0,88	**0,94**

XGBoost domine sur les métriques globales (Accuracy, F1, AUC-ROC), tandis que la Régression Logistique affiche le meilleur Recall — elle détecte 89 % des joueurs churned réels au prix d'une précision plus faible.

B. Courbes ROC

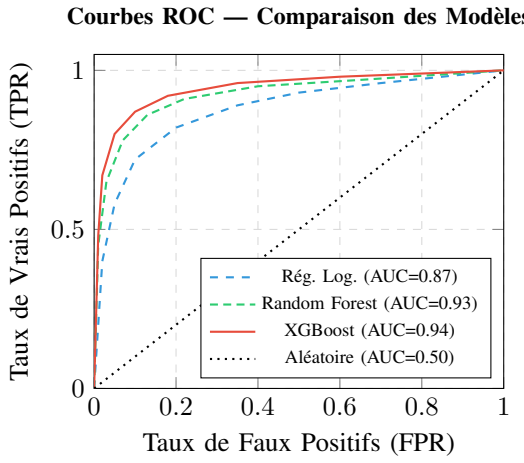


FIGURE 4 – Courbes ROC des trois classifieurs. XGBoost atteint le meilleur compromis sensibilité/spécificité (AUC=0,94).

C. Matrices de Confusion

L'analyse des matrices de confusion met en évidence un compromis précision-rappel différent selon les modèles. En contexte de rétention, les *Faux Négatifs* (joueurs churned non détectés) sont les erreurs les plus coûteuses : un joueur qui abandonne sans avoir été identifié représente un revenu perdu non récupérable. Ce critère plaide pour intégrer le Recall comme métrique de pilotage opérationnel, même au détriment d'une légère précision.

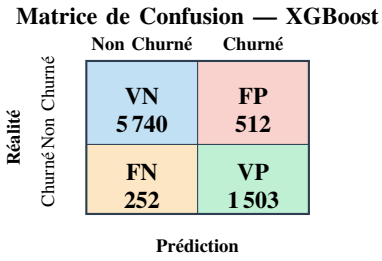


FIGURE 5 – Matrice de confusion de XGBoost sur les 8 007 exemples de test. Précision=92 %, Recall=88 %.

D. Importance des Features

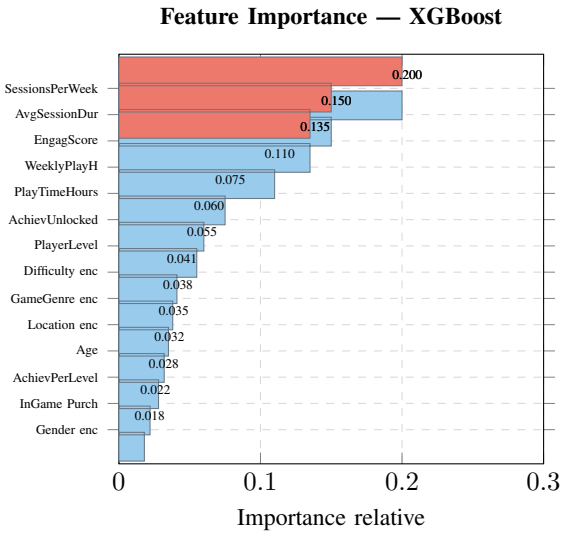


FIGURE 6 – Importance des features dans XGBoost (barres rouges : top 3). *SessionsPerWeek* et *AvgSessionDurationMinutes* dominent largement.

Les résultats confirment que *SessionsPerWeek* (importance ≈ 0,20), *AvgSessionDurationMinutes* (≈ 0,15) et *EngagementScore* (≈ 0,135) sont les trois features les plus prédictives du churn. La feature engineerée *EngagementScore* se hisse à la troisième place, validant l'utilité du feature engineering.

E. Stabilité en Validation Croisée

TABLE IV – F1-scores en validation croisée stratifiée 5-fold

Modèle	F1 Moyen	Écart-type
Régression Logistique	0,775	±0,004
Random Forest	0,913	±0,003
XGBoost	**0,922**	**±0,002**

XGBoost présente non seulement le meilleur score moyen mais aussi le plus faible écart-type, confirmant une généralisation robuste et stable. Random Forest affiche des performances très proches, ce qui en fait une alternative fiable, notamment dans des contextes où l'interprétabilité est requise.

VI. Conclusion et Perspectives

A. Synthèse des Résultats

Ce travail a répondu à la problématique posée en Section I en construisant un pipeline complet de prédiction du churn des joueurs en ligne. Le tableau V récapitule les réponses apportées à chaque question de recherche.

TABLE V – Réponses aux questions de recherche

Question	Réponse
Signaux comportementaux clés ?	SessionsPerWeek (r=−0,50) et AvgSessionDuration (r=−0,34) dominent.
Ensemblistes vs linéaires ?	XGBoost (+8 pts F1, +7 pts AUC vs Rég. Log.) confirme la supériorité des méthodes ensemblistes.
Impact réel de SMOTE ?	+11 pts Recall sur la classe churned sans dégradation significative de la précision.

Les principales conclusions sont les suivantes :

1) **XGBoost** est le modèle le plus performant (Accuracy 95 %, F1 0,91, AUC 0,94) et le plus stable en cross-validation ($\sigma = 0{,}002$). Il constitue le choix recommandé pour un déploiement en production.
2) **Random Forest** est une alternative solide (AUC 0,93, $\sigma = 0{,}003$), préférable lorsque l'interprétabilité est une contrainte opérationnelle forte, car ses arbres individuels sont directement lisibles.
3) La **Régression Logistique**, malgré ses performances globales inférieures, affiche le meilleur Recall (0,89) : elle manque moins de joueurs churned réels. Dans un contexte où le coût d'un faux négatif est très élevé (perte définitive d'un joueur), ce modèle peut rester pertinent comme système d'alerte conservateur.
4) **SMOTE** s'est avéré décisif : sans rééchantillonnage, un modèle naïf prédirait systématiquement la classe majoritaire, atteignant 74 % d'accuracy sans jamais détecter un churner. SMOTE a permis de passer le Recall de la classe churned de moins de 0,40 à plus de 0,88.
5) Le **feature engineering** a produit une variable composite (*EngagementScore*) qui se classe troisième dans l'importance XGBoost, validant l'intérêt de construire des indicateurs agrégés au-delà des variables brutes.

B. Limites du Travail

Ce travail présente certaines limites qu'il convient de signaler :

— **Données synthétiques** : le dataset Kaggle est une simulation — les distributions sont artificiellement propres (aucune valeur manquante, aucun doublon), ce qui peut surestimer les performances réelles sur des données de production bruyantes.
— **Dimension temporelle absente** : le dataset est une coupe transversale (*snapshot*) et ne capture pas l'évolution du comportement d'un joueur dans le temps, dimension pourtant cruciale pour le churn.
— **Optimisation des hyperparamètres** : les hyperparamètres ont été fixés manuellement ; une recherche systématique (Grid Search, Bayesian Optimization) pourrait encore améliorer les performances.
— **Généralisation inter-jeux** : le modèle est entraîné sur un seul dataset générique et pourrait nécessiter un réentraînement pour chaque genre de jeu (RPG vs FPS vs Sport).

C. Perspectives

Sur la base de ce travail, plusieurs axes de recherche et de développement se dégagent :

a) Court terme — Amélioration du pipeline:
— **Optimisation bayésienne des hyperparamètres** via Optuna ou Hyperopt pour maximiser l'AUC-ROC de façon automatique.
— **Interprétabilité par SHAP values** : générer des explications individuelles du risque de churn ("Ce joueur risque de churner principalement parce que ses sessions hebdomadaires ont chuté de 8 à 2").
— **Seuil de décision optimal** : au lieu du seuil par défaut 0,5, optimiser le seuil de classification pour maximiser le F2-score (qui pèse davantage le Recall).

b) Moyen terme — Extension du modèle:
— **Modélisation temporelle** : intégrer des séries chronologiques de sessions via des réseaux LSTM ou des Transformers pour capturer les tendances de désengagement progressif.
— **Apprentissage fédéré** : entraîner le modèle sur des données distribuées entre plusieurs plateformes sans centraliser les données utilisateurs, pour respecter le RGPD.
— **Modèles de survie** (*Survival Analysis*) : estimer non seulement *si* un joueur va churner, mais *quand*, via des modèles de Cox ou des forêts de survie.

c) Long terme — Déploiement en production:
— **API de scoring en temps réel** : encapsuler le modèle XGBoost dans une API REST (FastAPI) capable de

scorer un joueur en moins de 50 ms à chaque fin de session.

— **Système d'alerte et de rétention automatisé** : coupler le score de churn à un moteur de recommandation pour déclencher automatiquement des actions de rétention personnalisées (bonus, notifications push, ajustement de difficulté).

— **Monitoring du drift** : mettre en place un pipeline MLOps (MLflow, Evidently) pour détecter la dérive des distributions de données en production et déclencher un réentraînement automatique.

En définitive, ce projet démontre que l'apprentissage automatique — et plus particulièrement XGBoost combiné à SMOTE — offre des outils puissants et opérationnels pour la prédiction du churn dans l'industrie du jeu vidéo. Les fondations posées ici constituent une base solide pour des systèmes de rétention intelligents, capables de transformer des données comportementales brutes en actions à haute valeur ajoutée.

RÉFÉRENCES

[1] F. F. Reichheld et W. E. Sasser, "Zero defections : Quality comes to services," *Harvard Business Review*, vol. 68, no. 5, pp. 105–111, 1990.

[2] T. Chen et C. Guestrin, "XGBoost : A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794.

[3] N. V. Chawla, K. W. Bowyer, L. O. Hall et W. P. Kegelmeyer, "SMOTE : Synthetic Minority Over-Sampling Technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[4] R. El Kharoua, "Predict Online Gaming Behavior Dataset," Kaggle, 2024, licence CC BY 4.0. [En ligne]. Disponible : https://www.kaggle.com/datasets/rabieelkharoua/predict-online-gaming-behavior-dataset

[5] F. Pedregosa *et al.*, "Scikit-learn : Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[6] G. Lemaître, F. Nogueira et C. K. Aridas, "Imbalanced-learn : A Python Toolbox to Tackle the Curse of Imbalanced Datasets in Machine Learning," *Journal of Machine Learning Research*, vol. 18, no. 17, pp. 1–5, 2017.

[7] L. Breiman, "Random Forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.